Software Design Document (SDD)

Enterprise Multi-AI Agent Platform

1. Introduction

1.1 Purpose

This document details the architectural design, security posture, and data structures for the Enterprise Multi-AI Agent Platform. It serves as the authoritative blueprint for development, ensuring compliance with strict multi-tenancy requirements, secure tool execution, and scalable asynchronous processing.

1.2 Scope

The platform is a multi-tenant orchestration engine enabling users to define, deploy, and discover AI Agents. It features a Bring Your Own Key (BYOK) billing model, a visual Angular frontend, an event-driven FastAPI/Celery backend, and a WebAssembly-isolated Python sandbox for safe tool execution.

2. System Architecture (C4 Container Model)

The system utilizes an event-driven, decoupled topography to process long-running non-deterministic LLM operations without blocking the web gateway.

1. Client Tier (Angular 17+)

Utilizes NgRx and RxJS to manage the visual canvas state and parse incoming Server-Sent Events (SSE) representing agent thoughts.

2. API & Gateway Tier (FastAPI)

The sole ingress point. Handles asynchronous I/O, REST CRUD operations, and injects JWT tenant claims into the PostgreSQL session for database interactions.

3. Messaging Tier (Redis)

Dual functionality: Acts as the Task Broker queuing jobs for background workers, and as a Pub/Sub router bridging execution logs from workers back to the FastAPI WebSocket manager.

4. Orchestration Tier (Celery + LangGraph)

Horizontally scalable background worker fleet. LangGraph manages the cyclic LLM reasoning loops, injecting memory context and handling conditional sub-agent routing.

3. Security & Multi-Tenancy Architecture

Security operates on a "Zero Trust" model, ensuring that one tenant can never access or modify another tenant's agents, tools, or memory stores.

3.1 Authentication & Token Lifecycle

- **Enterprise SSO/OIDC:** Integration with Auth0/Keycloak to issue cryptographically signed JWTs.
- **Dual-Token Model:** Uses a short-lived (15 min) JWT Access Token embedded with custom claims (`org_id`, `space_ids`) to prevent repetitive DB lookups, alongside a 7-day HttpOnly, SameSite=Strict Refresh Token.

3.2 Attribute-Based Access Control (ABAC) & RLS

Application-level logic is reinforced by PostgreSQL Row-Level Security (RLS). Upon connection, FastAPI injects tenant variables into the DB session.

```
await db.execute(f"SET LOCAL app.current_org_id = '{jwt_org_id}';")
```

The database natively rejects queries that do not match the `org_id`, providing mathematical isolation.

3.3 Network Defense

- **Rate Limiting:** Token-bucket rate limiting applied via Redis at the Organization Level (`org_id`), preventing a single user from consuming the enterprise quota.
- **WebSocket Handshake:** Since WebSockets cannot securely pass auth headers, Angular requests a short-lived (10s TTL) ticket from the REST API to open the secure streaming pipe.

4. The Bring Your Own Key (BYOK) Model

To eliminate billing liability, tenants provide their own LLM API keys. Keys are never stored in plaintext.

1. **Ingestion:** FastAPI receives the key and instantly encrypts it using AES-256 via a Master Encryption Key (stored strictly as an environment variable).

2. **Storage:** The cipher text is saved in the `organization_secrets` table along with a masked preview (e.g., `sk-...8f9a`).
3. **Execution:** During a task run, the Celery worker retrieves and decrypts the key in memory, injecting it directly into the LangGraph model constructor. The variable is wiped from memory as soon as execution completes.

5. Execution Sandbox (Wasm Isolation)

> **Critical Threat Vector:** Allowing AI agents to execute dynamically generated Python tools introduces the risk of Remote Code Execution (RCE).

To mitigate this while maintaining performance on constrained infrastructure, standard Docker-in-Docker is avoided. Instead, tool execution is isolated using **WebAssembly (Wasm)** runtimes like Pyodide or Extism inside the Celery worker.

- **Zero Host Access:** Wasm modules have no access to the underlying OS filesystem or memory.
- **Network Deny-by-Default:** Prevents Server-Side Request Forgery (SSRF) and data exfiltration.
- **Microsecond Boot:** Significantly faster and lighter than container spinning.

6. Data & Persistence Model

Entity / Table	Core Attributes	Description & RLS Rules
spaces	id, org_id, access_level	Isolated boundary. Strict RLS policy enforcing org_id match.
agents	id, space_id, sys_prompt, visibility	visibility enum (PUBLIC/PRIVATE) allows cross-tenant cloning if PUBLIC.
tools	id, space_id, type, schema_json	Polymorphic design. Stores both Function definitions and Task instructions.
organization_secrets	org_id, encrypted_key, preview	Stores the AES-256 encrypted BYOK API credentials.

7. Requirement Analysis (NFRs)

- **Scalability:** Web tier (FastAPI) scales independently of the execution tier (Celery) based on traffic vs. compute demands.

- **Observability:** All LLM exceptions (AuthenticationError, RateLimitError) caught by Celery must be published back to the user via Redis Pub/Sub for immediate UI feedback.
- **Fault Tolerance:** If a Celery worker crashes mid-execution (OOM or kernel kill), the broker must retry the task, while the frontend displays a "Reconnecting" state via RxJS backoff.